

Projet DevOps · velos-api

BEREPION Elven — M2DAN26.1

Jalon 1 · Git

Dépôt initialisé avec le code fourni, travail sur branches (feature/route-sante, feature/documentation), conflit résolu, PR mergée, tag v1.0.0, branche main protégée.

C01 — Historique

Graphe des commits, branches, fusion et tag v1.0.0.

```
elven@DESKTOP-343K08J:~/velos-api$ git tag v1.0.0
git push origin v1.0.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: This repository moved. Please use the new location:
remote:   git@github.com:BerepionE/velos-api.git
To github.com:BerepionE/velos-api.git
 * [new tag]         v1.0.0 -> v1.0.0
elven@DESKTOP-343K08J:~/velos-api$ git log --oneline --graph --all
* a86e372 (HEAD -> main, tag: v1.0.0, origin/main, origin/HEAD) Nouveau test de protection
* ebf35a6 Test de la protection de branche
* 188789b Merge pull request #1 from BerepionE/feature/documentation
  \
   * 66e4e60 (origin/feature/documentation, feature/documentation) Ajoute le titre dans le README
  /
 * a6593c5 Resout le conflit de fusion
  \
   * 5b9f8b7 (feature/route-sante) Modifie le statut de sante (branche feature)
  /
 * f73fbf4 Modifie le statut de sante (branche main)
  \
   * 068c32b Initialise le projet avec le code fourni
elven@DESKTOP-343K08J:~/velos-api$
```

C02 — Conflit

Terminal au moment du merge conflictuel sur app.py.

```
elven@DESKTOP-343K08J:~/velos-api$ cat > app.py << EOF
'''velos-api : etat des stations de velos en libre-service.'''

import os
from flask import Flask, jsonify

app = Flask(__name__)

STATIONS_SECOURS = [
    {'nom': 'Gare Centrale', 'quartier': 'Centre', 'velos_disponibles': 12, 'capacite': 20},
    {'nom': 'Place du Marche', 'quartier': 'Centre', 'velos_disponibles': 2, 'capacite': 15},
    {'nom': 'Parc des Sports', 'quartier': 'Nord', 'velos_disponibles': 7, 'capacite': 10},
    {'nom': 'Universite', 'quartier': 'Sud', 'velos_disponibles': 1, 'capacite': 25},
]

def lire_stations():
    url = os.environ.get("DATABASE_URL")
    if not url:
        return STATIONS_SECOURS, "memoire"
    import psycopg2
    with psycopg2.connect(url) as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT nom, quartier, velos_disponibles, capacite FROM stations ORDER BY nom")
            lignes = cur.fetchall()
            stations = [{"nom": n, "quartier": q, "velos_disponibles": v, "capacite": c} for n, q, v, c in lignes]
    return stations, "postgres"

@app.get("/sante")
def sante():
    return jsonify({"statut": "ok"})

@app.get("/stations")
def stations():
    donnees, source = lire_stations()
    return jsonify({"source": source, "stations": donnees})

@app.get("/disponibilite")
def disponibilite():
    donnees, source = lire_stations()
    capacite_totale = sum(s["capacite"] for s in donnees)
    if not capacite_totale:
        return jsonify({"source": source, "taux_occupation": None})
    velos = sum(s["velos_disponibles"] for s in donnees)
    taux = round(100 * velos / capacite_totale, 1)
    return jsonify({"source": source, "taux_occupation": taux})

if __name__ == "__main__":
    port = int(os.environ.get("PORT", "8000"))
    app.run(host="0.0.0.0", port=port)
EOF

elven@DESKTOP-343K08J:~/velos-api$ git add
git commit -m "Initialise le projet avec le code fourni"
(main root-commit) 068c32b Initialise le projet avec le code fourni
4 files changed, 76 insertions(+)
create mode 100644 .gitignore
create mode 100644 app.py
create mode 100644 db/init.sql
create mode 100644 requirements.txt
elven@DESKTOP-343K08J:~/velos-api$ git checkout -b feature/route-sante
sed -i 's/"statut": "ok"/"statut": "en service"/' app.py
git add app.py
git commit -m "Modifie le statut de sante (branche feature)"

git checkout main
sed -i 's/"statut": "ok"/"statut": "operationnel"/' app.py
git add app.py
git commit -m "Modifie le statut de sante (branche main)"

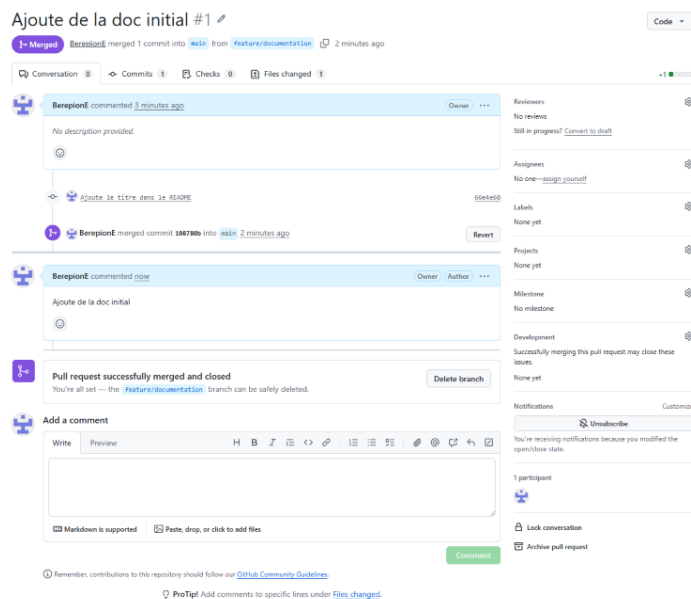
git merge feature/route-sante
Switched to a new branch 'feature/route-sante'
(feature/route-sante 5b9f8b7) Modifie le statut de sante (branche feature)
1 file changed, 1 insertion(+), 1 deletion(-)
Switched to branch 'main'
(main f73fbf4) Modifie le statut de sante (branche main)
1 file changed, 1 insertion(+), 1 deletion(-)
Auto-merging app.py
CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and then commit the result.
elven@DESKTOP-343K08J:~/velos-api$
```

Capture du fichier en conflit (partiellement illisible) :



C03 — Pull request

PR mergée sur GitHub.



C04 — Tag v1.0.0

Visible dans le graphe C01 : tag v1.0.0 sur le dernier commit main.

C05 — Protection de branche

Push direct sur main refusé (erreur GH006).

```
elven@DESKTOP-343K08J:~/velos-api$ echo "test numero 3" >> README.md
git add README.md
git commit -m "Test de blocage definitif"
git push
[main f61e179] Test de blocage definitif
1 file changed, 1 insertion(+)
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 316 bytes | 316.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: This repository moved. Please use the new location:
remote:  git@github.com:BerepionE/velos-api.git
remote: error: GH006: Protected branch update failed for refs/heads/main.
remote:
remote: - Changes must be made through a pull request.
To github.com:BerepionE/velos-api.git
 ! [remote rejected] main -> main (protected branch hook declined)
error: failed to push some refs to 'github.com:BerepionE/velos-api.git'
elven@DESKTOP-343K08J:~/velos-api$
```

Jalon 2 · Docker

C06 — Taille des images

```

elven@DESKTOP-343K08J:~/velos-api$ docker images | grep velos-api
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
velos-api:latest          5cbc5495530e          195MB          48MB
velos-api:naive           c2a234e64940          217MB          55.4MB
elven@DESKTOP-343K08J:~/velos-api$

```

C07 — Cache de build

[illegible]

C08 — Exécution non-root

```
e1ven@DESKTOP-343K08J:~/velos-api$ docker run --rm velos-api:latest whoami
appuser
e1ven@DESKTOP-343K08J:~/velos-api$
```

C09 — Pile démarrée

```

[+] Building 1.7s (16/19) FINISHED
[+] Internal: Load build name definitions
[+] reading from stdin/stdin
[+] Internal: Load build definition from Dockerfile
[+] transferring Dockerfile: 398B
[+] Internal: Load metadata for docker.io/library/python:3.12-slim
[+] Internal: Load dockergovernor
[+] Transferring context: 398B
[+] [Builder 4/4] PNM: docker.io/library/python:3.12-slim@sha256:7ab0870983cfe1a32d0e05b5cfc2f36608dc933362ef3c6cd008687d3a7
[+] Internal: Load dockergovernor: docker.io/library/python:3.12-slim@sha256:7ab0870983cfe1a32d0e05b5cfc2f36608dc933362ef3c6cd008687d3a7
[+] Internal: Load build context
[+] Internal: Load build file
[+] CACHED [Builder 5/4] WORKDIR /app
[+] CACHED [Stage 1/4] WORKDIR /app
[+] CACHED [Builder 6/4] COPY requirements.txt
[+] CACHED [Builder 6/4] RUN pip install --user -r requirements.txt
[+] CACHED [Stage 1/4] COPY --from=builder /root/.local /root/.local
[+] [Stage 1/4] CMD
[+] [Stage 1/4] Run docker --appuser /appuser /app
[+] exporting image
[+] exporting manifest: sha256:361606d6e7b57976f1d0b19129a7c8e61277f3182068a02561d0d0097ea2
[+] exporting manifest: sha256:361606d6e7b57976f1d0b19129a7c8e61277f3182068a02561d0d0097ea2
[+] exporting attestation manifest: sha256:776163c82b6e0e0e071ef2d10a76e58d81f257801f0a70e1b2a392292da
[+] exporting attestation manifest: sha256:776163c82b6e0e0e071ef2d10a76e58d81f257801f0a70e1b2a392292da
[+] naming to docker.io/library/wsls-api:latest
[+] naming to docker.io/library/wsls-api:latest
[+] resolving provenance for metadata file
[+]
wsls-api-api built
Network wsls-api-default Created
Volume wsls-api-db-data Created
Container wsls-api-db-1 healthy
Container wsls-api-api-1 started
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
wsls-api-api-1 wsls-api-api python app.py SERVICE
wsls-api-db-1 postgres:16-alpine "docker-entrypoint.s..." db 6 seconds ago Up Less than a second 5432/tcp

```

C10 — API servie par la base

```

#source: "http://www.velos.mcgill.ca/api/localhost/stations"
#source: "postgres://stations@[capacite]:12, nom="collège Jean Monin", quartier="Nord", velos_disponibles=8), [capacite]:20, nom="dare Centrale", quartier="Centre", velos_disponibles=12), [capacite]:19, nom="dare des Arts", quartier="Nord", velos_disponibles=7), [capacite]:18, nom="Polytec Glyssiane", quartier="Sud", velos_disponibles=14), [capacite]:19, nom="Polytec de la Marche", quartier="Centre", velos_disponibles=21), [capacite]:25, nom="Université", quartier="Sud", velos_disponibles=11), [capacite]:18, nom="Zoo et Botanique", quartier="Ouest", velos_disponibles=5]]

```

C11 — Persistence

Station 'Test Persistence' ajoutée, puis docker compose down/up : toujours présente.

```
elven@DESKTOP-343K08J:~/velos-api$ curl http://localhost:8080/stations
{"source":"postgres","stations":[{"capacite":12,"nom":"College Jean Moulin","quartier":"Nord","velos_disponibles":0},{"capacite":20,"nom":"Gare Centrale","quartier":"Centre","velos_disponibles":12},{"capacite":18,"nom":"Métro de Ville","quartier":"Centre","velos_disponibles":9},{"capacite":10,"nom":"Place des Sources","quartier":"Nord","velos_disponibles":77},{"capacite":16,"nom":"Placine Olympique","quartier":"Sud","velos_disponibles":140},{"capacite":15,"nom":"Place du Marché","quartier":"Centre","velos_disponibles":23},{"capacite":18,"nom":"Test Persistence","quartier":"Est","velos_disponibles":50},{"capacite":25,"nom":"Université","quartier":"Sud","velos_disponibles":11},{"capacite":18,"nom":"Zone Industrielle","quartier":"Ouest","velos_disponibles":0}]}
elven@DESKTOP-343K08J:~/velos-api$
```

Image publiée

docker.io/eberepion/velos-api:1.0

```
elven@DESKTOP-343K08J:~/velos-api$ docker tag velos-api:latest eberepion/velos-api:1.0
docker push eberepion/velos-api:1.0
The push refers to repository [docker.io/eberepion/velos-api]
717bec52f794: Pushed
b79f58b3eb94: Mounted from eberepion/meteo-api
f99d5186ca5c: Pushed
0542374765af: Pushed
07ce962e515f: Mounted from eberepion/meteo-api
9fc0809aefc5: Pushed
561f996e2284: Pushed
6310eb16bf42: Mounted from eberepion/meteo-api
48579d87405c: Mounted from eberepion/meteo-api
a0d8783128af: Mounted from eberepion/meteo-api
1.0: digest: sha256:5d6db5bb671633d89fee9dc7966aabca0ff24bfdab51805aa06e20e21998d2e size: 856
elven@DESKTOP-343K08J:~/velos-api$
```

Jalon 3 · Kubernetes

Cluster Kind (1 control-plane, 2 workers). Base et API déployées, secret K8s pour le mot de passe, ConfigMap pour init.sql, port 8081 obtenu via port-forward, route /alertes ajoutée et publiée en 2.0, mise à jour sans coupure puis retour arrière démontrés.

C12 — Cluster opérationnel

kubectl get nodes : 3 nœuds Ready (devops-control-plane, devops-worker, devops-worker2).

```
elven@DESKTOP-343K08J:~/velos-api$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
devops-control-plane               Ready    control-plane   44m   v1.37.0-rc.1
devops-worker                      Ready    <none>         44m   v1.37.0-rc.1
devops-worker2                    Ready    <none>         44m   v1.37.0-rc.1
elven@DESKTOP-343K08J:~/velos-api$
```

Secret et ConfigMap

Mot de passe créé directement dans le cluster (kubectl create secret), jamais dans un fichier commité. Jeu de données initial injecté via un ConfigMap (kubectl create configmap init-sql --from-file=db/init.sql).

C13 — Ressources déclarées

Pods, services et déploiements : API (2/2) et DB (1/1) Running.

```
elven@DESKTOP-343K08J:~/velos-api$ kubectl get pods,svc,deploy
NAME                                READY    STATUS    RESTARTS   AGE
pod/api-deployment-f846f575f-fsttm 1/1      Running   0           37s
pod/api-deployment-f846f575f-plch2 1/1      Running   0           37s
pod/db-deployment-7844b5d4b8-98jwh 1/1      Running   0          111s
pod/meteo-api-677f5c9548-vlqkh      1/1      Running   1 (33m ago) 24h
pod/meteo-db-5bccbd8699-wjj9v       1/1      Running   2 (33m ago) 44h

NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP    PORT(S)          AGE
service/api-service                 ClusterIP          10.96.16.7      <none>         8080/TCP          37s
service/db-service                  ClusterIP          None            <none>         5432/TCP          111s
service/kubernetes                  ClusterIP          10.96.0.1       <none>         443/TCP           44h
service/meteo-api                   NodePort           10.96.166.11    <none>         8080:30080/TCP   44h
service/meteo-db                    ClusterIP          10.96.241.213   <none>         5432/TCP          44h

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/api-deployment      2/2      2              2            37s
deployment.apps/db-deployment        1/1      1              1           111s
deployment.apps/meteo-api            1/1      1              1            24h
deployment.apps/meteo-db             1/1      1              1            44h
elven@DESKTOP-343K08J:~/velos-api$
```

C14 — API servie par le cluster

Port-forward vers 8081, GET /stations → source: postgres.

```
elven@DESKTOP-343K08J:~/velos-api$ curl http://localhost:8081/stations
{"source":"postgres","stations":[{"capacite":12,"nom":"College Jean Moulin","quartier":"Nord","velos_disponibles":0},{"capacite":20,"nom":"Gare Centrale","quartier":"Centre","velos_disponibles":12},{"capacite":18,"nom":"Métro de Ville","quartier":"Centre","velos_disponibles":9},{"capacite":10,"nom":"Place des Sources","quartier":"Nord","velos_disponibles":77},{"capacite":16,"nom":"Placine Olympique","quartier":"Sud","velos_disponibles":140},{"capacite":15,"nom":"Place du Marché","quartier":"Centre","velos_disponibles":23},{"capacite":18,"nom":"Test Persistence","quartier":"Est","velos_disponibles":50},{"capacite":25,"nom":"Université","quartier":"Sud","velos_disponibles":11},{"capacite":18,"nom":"Zone Industrielle","quartier":"Ouest","velos_disponibles":0}]}
elven@DESKTOP-343K08J:~/velos-api$
```

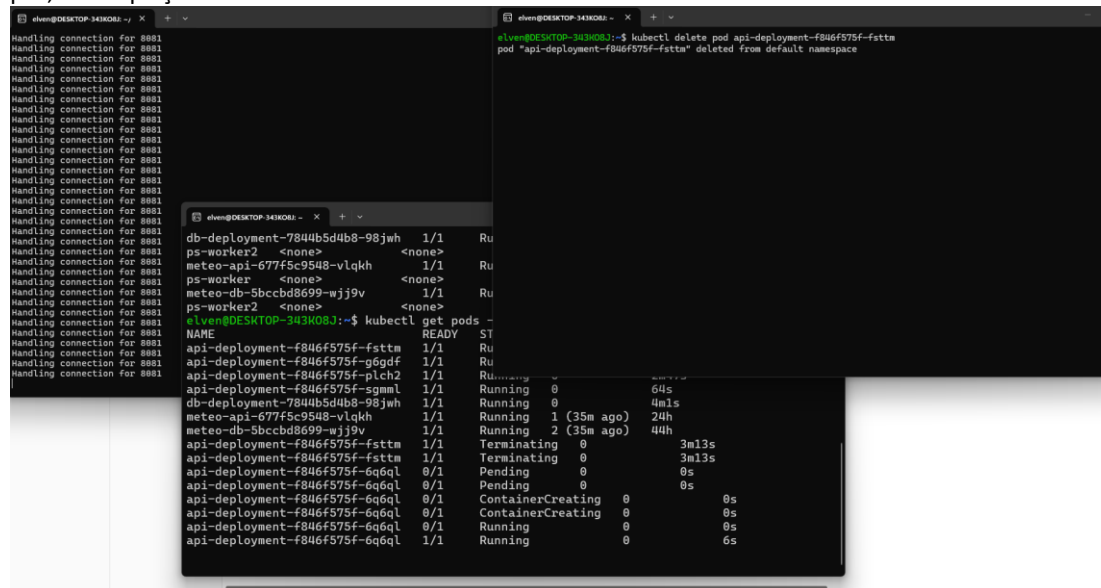
C15 — Passage à l'échelle

4 exemplaires de l'API, répartis entre devops-worker et devops-worker2.

```
elven@DESKTOP-343K08J:~/velos-api$ kubectl get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE   IP              NODE              NOMINATED NODE    READINESS GATES
api-deployment-f846f575f-fsttm      1/1      Running   0           2m6s  10.244.1.3      devops-worker     <none>             <none>
api-deployment-f846f575f-ggdf       1/1      Running   0           23s   10.244.1.4      devops-worker     <none>             <none>
api-deployment-f846f575f-plch2      1/1      Running   0           2m6s  10.244.2.4      devops-worker2    <none>             <none>
api-deployment-f846f575f-sgml       1/1      Running   0           23s   10.244.2.5      devops-worker2    <none>             <none>
db-deployment-7844b5d4b8-98jwh      1/1      Running   0           3m28s  10.244.2.3      devops-worker2    <none>             <none>
meteo-api-677f5c9548-vlqkh          1/1      Running   1 (34m ago) 24h    10.244.1.2      devops-worker     <none>             <none>
meteo-db-5bccbd8699-wjj9v           1/1      Running   2 (34m ago) 44h    10.244.2.2      devops-worker2    <none>             <none>
elven@DESKTOP-343K08J:~/velos-api$
```

C16 — Résistance à la panne

Suppression d'un pod pendant que le trafic passe (boucle de curl sur 8081 à droite) : le service ne s'interrompt pas, le remplaçant redémarre seul.

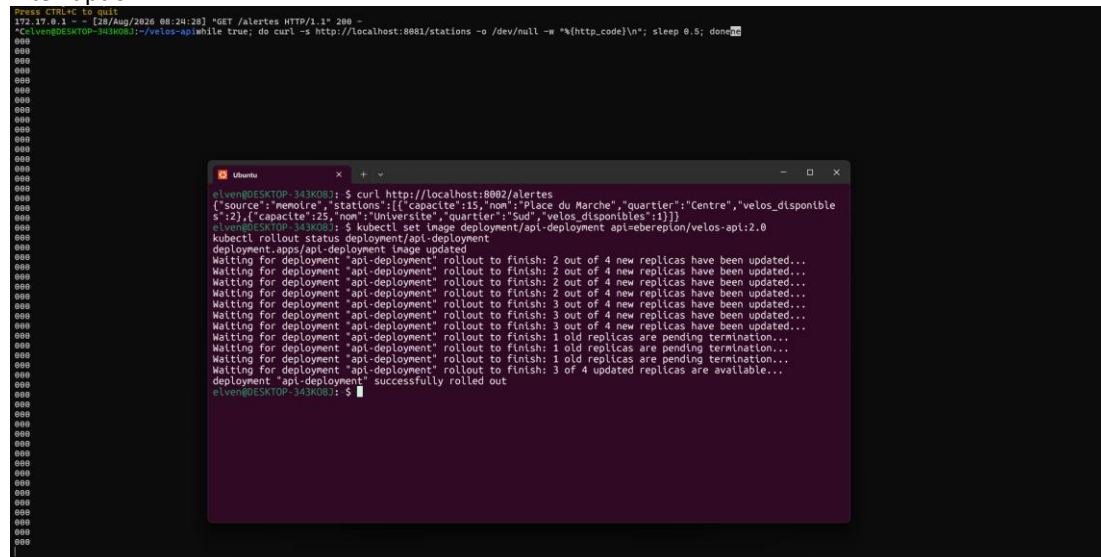


Évolution en version 2.0

Route /alertes ajoutée (stations avec `velos_disponibles` ≤ 2), testée en local, image publiée `eberepion/velos-api:2.0`.

C17 — Mise à jour sans coupure

kubectl set image vers la 2.0 pendant qu'une boucle de curl sur 8081 continue de répondre 200 sans interruption.



C18 — Retour arrière

kubectl rollout undo : historique des révisions, puis vérification que les 4 pods API sont repassés en 1.0.

```

$@ven@DESKTOP-343M08J:~/velos-api$ kubectl rollout undo deployment/api-deployment
kubectl rollout status deployment/api-deployment
kubectl rollout history deployment/api-deployment
Warning: resource deployment/api-deployment was previously managed with 'kubectl apply'. Rolling back will not update the kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply' operations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/api-deployment rolled back
Waiting for deployment 'api-deployment' rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment 'api-deployment' rollout to finish: 1 old replicas are pending termination...
Waiting for deployment 'api-deployment' rollout to finish: 1 old replicas are pending termination...
Waiting for deployment 'api-deployment' rollout to finish: 1 old replicas are pending termination...
Waiting for deployment 'api-deployment' rollout to finish: 3 of 4 updated replicas are available...
deployment 'api-deployment' successfully rolled out
deployment.apps/api-deployment
revision: 2
changes: 2
2
3
<none>
<none>

$@ven@DESKTOP-343M08J:~/velos-api$ kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}'
eberepion/velos-api:2.0 eberepion/velos-api:2.0 eberepion/velos-api:2.0 eberepion/velos-api:1.0 eberepion/velos-api:1.0 eberepion/velos-api:1.0 eberepion/velos-api:1.0 postgres:16-alpine
eberepion/meteo-api:8 postgres:16-alpine|ven@DESKTOP-343M08J:~/velos-api$
$@ven@DESKTOP-343M08J:~/velos-api$ kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}'
eberepion/velos-api:1.0 eberepion/velos-api:1.0 eberepion/velos-api:1.0 eberepion/velos-api:1.0 postgres:16-alpine eberepion/meteo-api:8 postgres:16-alpine|ven@DESKTOP-343M08J:~/velos-api$

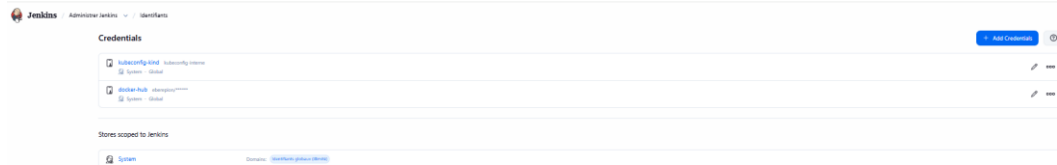
```

Jalon 4 · Jenkins

Tests automatisés (2, sans base de données) intégrés dans un étage dédié du Dockerfile. Pipeline versionné (Jenkinsfile) avec 4 étapes enchaînées : Tester, Construire, Publier, Déployer. Identifiants Docker Hub et kubeconfig gérés via le gestionnaire d'identifiants Jenkins. Déclenchement automatique par push (Poll SCM). Démonstration du rouge utile : test cassé volontairement, pipeline bloqué avant publication/déploiement, puis réparation et retour au vert.

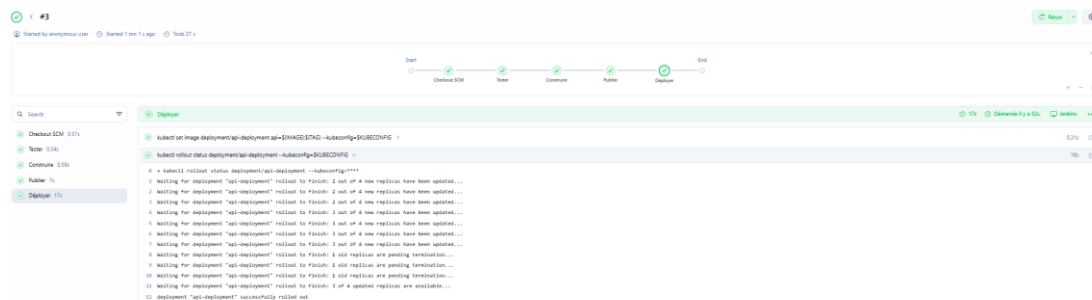
C19 — Identifiants protégés

Liste des identifiants Jenkins (docker-hub, kubeconfig-kind), aucune valeur en clair visible.



C20 — Pipeline vert

Les 4 étapes au vert (Checkout, Tester, Construire, Publier, Déployer), avec le détail du rollout attendu jusqu'à la fin réelle.



C21 — Déclenchement automatique

Console du build : "Started by an SCM change" — déclenché par le Poll SCM après un git push, sans clic manuel.

```

Started by an SCM change
Obtained Jenkinsfile from git https://github.com/eberepion/velos-api.git
[Pipeline] Start: on Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/velos-api-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/velos-api-pipeline/.git # timeout:10

```

C22 — Le rouge utile

The screenshot shows the GitHub Actions workflow interface for the repository 'lucioalmeida/terraform-provider-oci'. At the top, a status bar indicates 'Lancé par un changement dans le code' (Triggered by a code change), 'Started 2 min 54 s ago', and 'Task 5.7.1'. Below this, a workflow diagram shows the sequence of steps: 'Start' (green circle), 'Checkout SCM' (green circle), 'Setup' (red circle with a white 'X'), 'Configure' (blue circle), 'Publish' (blue circle), 'Démarrer' (blue circle), and 'End' (blue circle). The 'Setup' step is currently selected, and its details are shown in the main panel below. The details panel has a red header with the title 'Setup' and a red background. It contains a list of steps with a search bar and a filter icon. The first step is 'docker build --target tester <ref>api-test:...' (blue header). The second step is 'docker build --target tester <ref>api-test:...' (blue header). The third step is 'SERVICATO: The legacy builder is deprecated and will be removed in a future release.' (blue header). The fourth step is 'Install the builder component to build images with BuildKit:' (blue header). The fifth step is 'https://docs.docker.com/get/buildkit/' (blue header). The sixth step is 'Sending build context to Docker daemon 18.94kB' (blue header). The seventh step is 'Step 1/1 : FROM scratch:3.12-slim AS builder' (blue header).

Extrait du journal : AssertionError sur test_alertes_seuil, ligne 17 du test.

```

test_app.py::test_sar PASSED [ 50%]
test_app.py::test_alertes_seuil FAILED [100%]

===== FAILURES =====
_____ test_alertes_seuil _____

def test_alertes_seuil():
    client = app.test_client()
    reponse = client.get("/alertes")
    assert reponse.status_code == 200
    donnees = reponse.get_json()
    > assert all(s["velos_disponibles"] <= 0 for s in donnees["stations"]) # cassé e
    + where False = all(<generator object test_alertes_seuil.<locals>.<genexpr> at 0x7f9b1b1b1b1b>)
    + where False = all(<generator object test_alertes_seuil.<locals>.<genexpr> at 0x7f9b1b1b1b1b>)

test_app.py:17: AssertionError
===== short test summary info =====
FAILED test_app.py::test_alertes_seuil - assert False
===== 1 failed, 1 passed in 0.14s =====

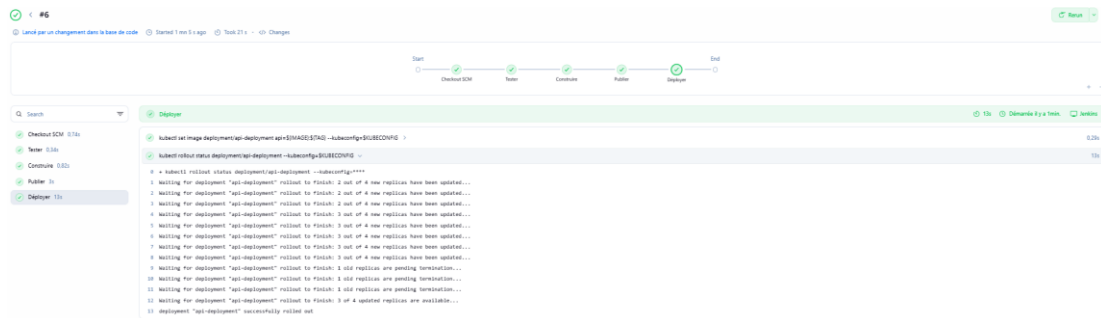
The command 'bin/sh -c python -m pytest test_app.py -v' returned a non-zero code: 1
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Construire)
Stage "Construire" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Publier)
Stage "Publier" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Déployer)
Stage "Déployer" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE

```

Les 4 pods API tournent sur eberepion/velos-api:3, l'image produite et poussée par ce build Jenkins (pas déployée à la main).

Réparation et retour au vert

Test corrigé, nouveau push, pipeline entièrement vert (build #6), déploiement réussi.



Jalon 5 · Rapport et preuves

1. Ce que j'ai construit, en cinq lignes

J'ai repris le code fourni de l'API vélos et je l'ai rendu livrable de bout en bout. Le code est versionné sur GitHub avec un historique propre et une branche main protégée. L'application est conteneurisée avec Docker (image multi-stage, non-root), et tourne en local avec une base PostgreSQL via docker-compose. Elle est ensuite déployée dans un cluster Kubernetes (Kind) avec 4 instances de l'API, secret pour le mot de passe, et scaling/rolling update testés. Enfin, un pipeline Jenkins automatise tout : dès qu'on pousse du code, il teste, construit l'image, la publie sur Docker Hub et déploie automatiquement dans le cluster.

2. Le trajet d'une requête

Quand j'appelle `http://localhost:8081/stations` depuis mon navigateur, la requête passe d'abord par le kubectl port-forward qui redirige le port 8081 de ma machine vers le port 8000 du service api-service dans le cluster. Le service Kubernetes fait ensuite du load balancing entre les 4 pods de l'API (via les labels `app: api`). Le pod choisi exécute le code Flask, qui appelle `lire_stations()` : cette fonction lit la variable `DATABASE_URL` et se connecte au service db-service (résolu par le DNS interne du cluster), qui pointe vers le pod PostgreSQL. La requête SQL est exécutée, les résultats remontent jusqu'au pod API, qui les transforme en JSON, qui remonte à travers le service, le port-forward, et finit dans mon navigateur.

3. Mes trois difficultés

- 1) L'API plantait au démarrage dans Docker avec une `IndentationError` sur la ligne 29 : des marqueurs de conflit Git (`<<<<<<< HEAD`) étaient restés dans `app.py` après une fusion mal terminée au jalon 1. Correction : suppression des marqueurs, garde du bon code, nouveau commit.
- 2) Le push Docker Hub échouait avec "insufficient scopes" puis "incorrect username or password" : le token d'accès généré avait un scope en lecture seule, et un mot de passe classique ne fonctionne plus pour les logins programmatiques. Correction : génération d'un nouveau token avec le scope "Read & Write", mis à jour dans les identifiants Jenkins.
- 3) Le pipeline Jenkins ne se déclenchait jamais tout seul après un git push : la case "Scrutation de l'outil de gestion de version" était cochée mais le champ de planning était resté vide. Correction : ajout du planning `* * * *` pour scruter le dépôt toutes les minutes.

4. Ce qui n'est pas fait

Je n'ai pas ajouté de sonde de liveness en plus de la sonde de disponibilité (readiness) sur le déploiement de l'API — je n'ai eu que la readiness probe sur `/sante`. Je n'ai pas non plus déclaré de limites de ressources (CPU/mémoire) sur les conteneurs du cluster, faute de temps. Sur certaines captures, j'ai eu des soucis de mise en page une fois enregistrées (fenêtres mal cadrées ou perdues), et il était trop tard pour refaire exactement le même scénario (par exemple un rollout ou une suppression de pod déjà terminée), donc certaines captures ne sont pas aussi propres que je l'aurais voulu.

5. Assistance utilisée

J'ai utilisé Claude surtout pour la mise en page du rapport Word, pour que ce soit propre et bien organisé, n'étant pas très doué sur ce plan. Je m'en suis aussi servi de temps à autre quand je restais bloqué : pour

comprendre certains messages d'erreur (conflit Git non résolu, token Docker Hub, Poll SCM mal configuré), et pour identifier la cause exacte d'un problème avant de trouver moi-même la correction.